

Software Engineering Lab - Lab 3

Component Modelling & Architectural Pattern Selection

Arjun Srikanth | PES1UG24AM920 | Section H
Problem Statement #48 - Incident Escalation & On-Call Rotation Engine

1. Architecture Selection

Selected pattern: Event-driven microservices architecture.

2. Requirements Considered

- Alert dispatch should start within 3 seconds.
- Unacknowledged incidents must escalate after the configured timeout (5 minutes in the base requirement).
- Engineers should be able to acknowledge or resolve incidents through more than one channel.
- The system should remain highly available and protect incident/contact data.

3. Short Pattern Comparison

Pattern	What is useful	Main issue for this system
Layered	Simple structure and easy to understand.	Harder to scale alert ingestion and notifications independently.
Client-Server	Central server keeps logic in one place.	The central server can become a bottleneck or failure point.
Event-driven microservices	Services can work independently and communicate through events.	More deployment complexity, but it fits bursty alert traffic better.

4. Justification

Reason 1 - burst handling: Alert ingestion is separated from notification. If an SMS/email provider is slow, new alerts can still be accepted and queued.

Reason 2 - independent scaling: Ingestion and notification can need more capacity than rotation management, so only the busy services need to scale.

Security advantage: The public webhook can be isolated from internal services. Incoming alerts can be authenticated, internal connections encrypted, and contact details limited to the notification service.

Performance benefit: The ingestion service can place an event on a queue and return quickly while escalation and notification continue in parallel. This supports the 3-second alert-dispatch requirement from Lab 1.

The design is kept at component level only; vendor-specific deployment details are intentionally not included.

5. UML Component Diagram

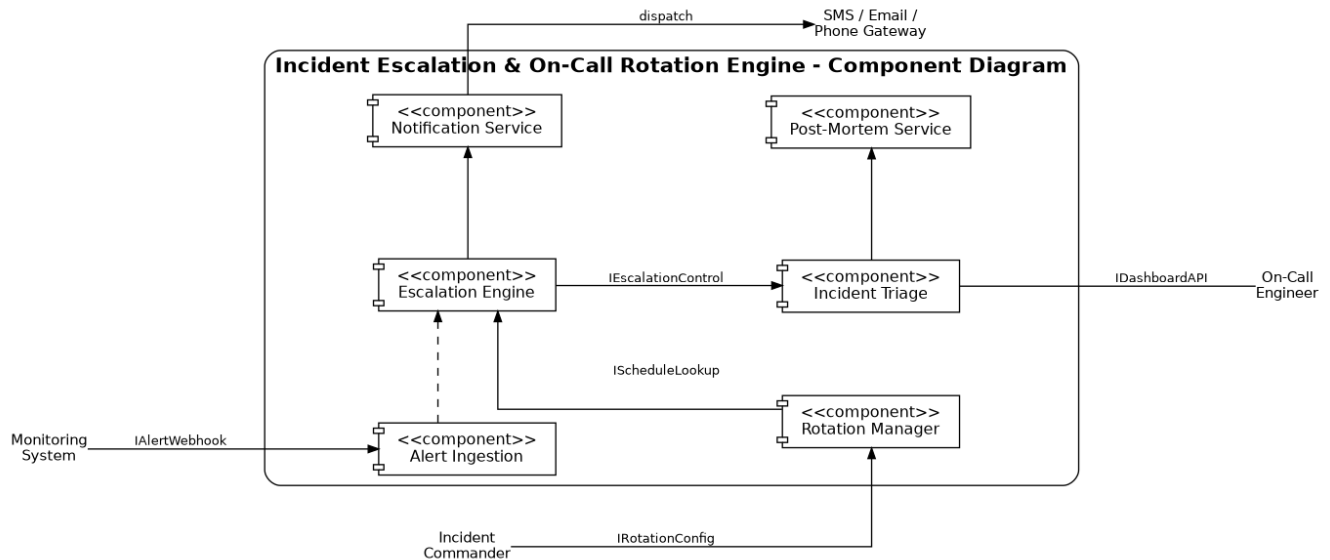


Figure 1. Six components with clearer connections and named interfaces.

6. Components and Interfaces

Component	Responsibility	Provided Interface(s)	Required Interface(s)
Alert Ingestion	Receives and validates monitoring alerts.	IAlertWebhook	-
Rotation Manager	Maintains schedules and finds the active engineer.	IScheduleLookup, IRotationConfig	-
Escalation Engine	Applies timeouts and escalation rules.	IEscalationControl	IScheduleLookup, INotifyDispatch
Notification Service	Sends SMS, email or phone notifications.	INotifyDispatch	-
Incident Triage	Handles acknowledge and resolve actions.	IDashboardAPI	IEscalationControl, IPostMortemTrigger
Post-Mortem Service	Stores the incident summary after resolution.	IPostMortemTrigger	-

Deliverable check: 6 components and 6 named interfaces are shown, with cleaner connectivity between the main components.